

**A PROJECT STAGE-I REPORT
ON**

“PENETRATION TESTING FOR WEBSITES USING LLM”



**Submitted to
SAVITRIBAI PHULE PUNE UNIVERSITY
In Partial Fulfilment of the Requirement for the Award of
BACHELOR OF ENGINEERING
IN
INFORMATION TECHNOLOGY
BY**

**“Pallavi Akolkar [IT3002]”
“Soham Deshmukh [IT3011]”
“Mohanish Kulkarni [IT3032]”
“Jayam Mehta [IT3054]”**

**Under the guidance of
“Dr.Vilas Gaikwad”**



TRINITY

**DEPARTMENT OF INFORMATION TECHNOLOGY
KJ's EDUCATIONAL INTSITUTE
TRINITY COLLEGE OF ENGINEERING AND RESEARCH
Haveli Pisoli Annex, Pune - 411048
2024-2025**

CERTIFICATE



This is to certify that the Project Report entitled
“Penetration Testing For Websites Using LLM”

submitted by

Pallavi Akolkar (IT4002)

Soham Deshmukh (IT4011)

Mohanish Kulkarni (IT4032)

Jayam Mehta (IT4054)

*is a bonafide work carried out by them under the supervision and guidance of **Dr. V. S. Gaikwad**, and it is approved for partial fulfillment of the requirement for the **BE (Information Technology Engineering)** course of Savitribai Phule Pune University for the award of the Degree of Bachelor of Engineering (Information Technology Engineering).*

This project report has not been earlier submitted to any other institute or university for the award of any degree or diploma.

Dr. V. S. Gaikwad
Internal guide

Dr. V. S. Gaikwad
Head of Department (IT)

External Examiner

Dr. Abhijeet Auti
Principal (TCOER,Pune)

ACKNOWLEDGMENT

We would like to take this opportunity to express our gratitude towards all the people who have in various ways helped in the successful completion of our project. We must convey our gratitude to **Dr. Vilas Gaikwad** for giving us the constant source of inspiration, personally correcting our work, and providing encouragement throughout the project.

We express deep gratitude towards **Dr. Vilas Gaikwad**, Head of the Information Technology Engineering Department, for providing support and giving his valuable time.

We also express our gratitude towards our **father, mother, other family members, and our friends** for encouraging us with their valuable suggestions and motivating us from time to time.

Group No. PR2025-26.01

Name of Students

Sign.

- 1) Pallavi Akolkar**
- 2) Soham Deshmukh**
- 3) Mohanish Kulkarni**
- 4) Jayam Mehta**

DECLARATION

We, the undersigned, hereby declare that the Project entitled “**Penetration Testing For Websites Using LLM**” submitted by us to **Trinity College of Engineering and Research, Pune**, for the award of the degree of **Bachelor of Engineering in Information Technology Engineering**, under the guidance of **Dr. Vilas Gaikwad** is our original work.

We further declare that to the best of our knowledge and belief, this work has not been previously submitted to this or any other university.

Place:

Date:

Group No.

Name of Students

Sign.

- 1) Pallavi Akolkar**
- 2) Soham Deshmukh**
- 3) Mohanish Kulkarni**
- 4) Jayam Mehta**

ABSTRACT

Web-facing applications are increasingly targeted by sophisticated attackers, while traditional vulnerability scans rely on static rules and manual expertise that can miss complex, logic- and context-dependent flaws. This project proposes an LLM-augmented Penetration Testing framework for websites that combines automated reconnaissance, dynamic scanning, fuzzing, and human-in-the-loop validation with large language models to improve discovery, exploitation reasoning, and report generation. The system uses LLMs to (1) synthesize reconnaissance data into prioritized hypotheses, (2) generate adaptive attack payloads for fuzzing and injection tests, (3) infer application-specific attack chains (including business-logic and chained vulnerabilities), and (4) draft clear, actionable remediation guidance. Safety and ethics are enforced with guardrails: scope constraints, rate limits, and exploit-sanitization to avoid destructive actions. A feedback loop lets security analysts confirm or refine LLM-suggested findings, reducing false positives while accelerating triage. Implemented using modular scanners, an orchestration engine, payload generation and validation modules, and secure audit logging, the framework was evaluated in controlled testbeds and real-world staging environments against OWASP Top 10 scenarios. Results show improved coverage of complex vulnerabilities, faster triage, and higher-quality reports—demonstrating that LLM-augmented pentesting can enhance analyst productivity, reduce risk, and scale responsible security testing.

Contents

1	INTRODUCTION	1
1.1	Overview	1
1.2	Domain	1
1.3	Purpose	1
1.4	Scope	1
1.5	Motivation	2
1.6	Objective of the Project	2
2	LITERATURE REVIEW	3
3	WORKFLOW OF PROJECT	6
4	GENERAL DESCRIPTION	7
4.1	What is an Automated Penetration Testing System using LLM?	7
4.2	What is LLM-assisted Penetration Testing for Websites?	7
4.3	What is Automation Orchestration in this context?	8
4.4	Role of AI / LLMs in Web Penetration Testing	9
4.5	System Characteristics	9
4.6	Typical Components / Modules	10
4.7	Benefits	10
4.8	Limitations Risks	10
4.9	Safe Implementation Guidelines	11
4.10	Future Enhancements	11
5	ARCHITECTURE OF SYSTEM	12
5.1	Hardware Requirements	12
5.2	Software Requirements	12
5.3	System Workflow	13
6	TEST CASES	18
7	EXPERIMENTAL RESULTS	19
8	PROJECT PLAN	20
9	APPLICATIONS AND ADVANTAGES	21
9.1	Applications of LLM-Based Penetration Testing System	21
9.2	Advantages of LLM-Based Penetration Testing	21
9.3	Limitations of the System	21
10	DISCUSSION AND CONCLUSION	23
11	REFERENCES	24
	Appendix-A: Plagiarism Report	25

Appendix-B: Base Paper(s)	26
Appendix-C: Tools Used / Software Components Specifications	27
Appendix-D: Published Papers and Certificates	28
Appendix-E: Reviews Evaluation Sheet	29

List of Figures

1	Workflow of automated PT	7
2	Conceptual Representation of System	8
3	Hardware Architecture of System	13
4	Software Architecture and Data Flow	14
5	System Workflow	18
6	Plagiarism Report	25
7	Paper Acceptance	28
8	Review Evaluation 1	30
9	Review Evaluation 2	31
10	Review Evaluation 3	32

List of Tables

1	Literature Review	5
2	Detailed Test Cases for LLM-based Penetration Testing System	19
3	Schedule of Project Stage-I Work	20

List of Abbreviations

- **AI** – Artificial Intelligence
- **API** – Application Programming Interface
- **CISS** – Cyber Intelligence and Security Systems (IEEE Conference)
- **CI** – Continuous Integration
- **CIA** – Confidentiality, Integrity, and Availability
- **CI/CD** – Continuous Integration / Continuous Deployment
- **CMS** – Content Management System
- **CVE** – Common Vulnerabilities and Exposures
- **CVSS** – Common Vulnerability Scoring System
- **DAST** – Dynamic Application Security Testing
- **DBMS** – Database Management System
- **DFD** – Data Flow Diagram
- **GUI** – Graphical User Interface
- **HTML** – HyperText Markup Language
- **HTTP** – HyperText Transfer Protocol
- **IDOR** – Insecure Direct Object Reference
- **IEEE** – Institute of Electrical and Electronics Engineers
- **IRJET** – International Research Journal of Engineering and Technology
- **JS** – JavaScript
- **JSON** – JavaScript Object Notation
- **LLM** – Large Language Model
- **ML** – Machine Learning
- **MITRE ATT&CK** – MITRE Adversarial Tactics, Techniques, and Common Knowledge
- **NLP** – Natural Language Processing
- **Nmap** – Network Mapper
- **OWASP** – Open Worldwide Application Security Project
- **PDF** – Portable Document Format
- **PII** – Personally Identifiable Information

- **PoC** – Proof of Concept
- **PR** – Pull Request
- **REST** – Representational State Transfer
- **RL** – Reinforcement Learning
- **RoE** – Rules of Engagement
- **SAST** – Static Application Security Testing
- **SOC** – Security Operations Center
- **SQL** – Structured Query Language
- **SQLi** – SQL Injection
- **SPA** – Single Page Application
- **SSRF** – Server-Side Request Forgery
- **SRS** – Software Requirements Specification
- **TCOER** – Trinity College of Engineering and Research
- **UI** – User Interface
- **UML** – Unified Modeling Language
- **URL** – Uniform Resource Locator
- **VS Code** – Visual Studio Code
- **XSS** – Cross-Site Scripting
- **ZAP** – Zed Attack Proxy (OWASP)

Trinity College of Engineering and Research

Department of Information Technology

Project Synopsis

Project Group Number: PR 2025-26_01

Project Title: Penetration Testing for Websites Using LLM

Name of the Students : Mr. Soham Deshmukh

Mr. Mohanish Kulkarni

Miss. Pallavi Akolkar

Mr. Jayam Meheta

Name of the Guide : Dr. Vilas Gaikwad

Broad Area : Information Forensics and Security

1 Introduction

With the increasing adoption of web technologies, the attack surface for cyber threats is expanding significantly. Modern web applications are complex ecosystems containing critical business logic, sensitive data transactions, and numerous third-party integrations, making them attractive and high-value targets for malicious actors. Traditional penetration testing provides a structured approach to identifying vulnerabilities, but as a largely manual process, it is often resource-intensive, time-consuming, and difficult to scale across large enterprise environments.

The emergence of Artificial Intelligence (AI) and Machine Learning (ML) has introduced a new paradigm in cybersecurity. These technologies offer the potential to automate and enhance threat detection and analysis. This project leverages the advanced reasoning and natural language capabilities of Large Language Models (LLMs) to augment and automate penetration testing workflows. By automating the generation of context-aware attack payloads, dynamically analyzing application responses, and providing human-readable explanations for discovered vulnerabilities, this project introduces an intelligent, scalable, and explainable penetration testing framework tailored for the complexities of modern web applications. The goal is to bridge the gap between the exhaustive but slow manual testing process and the fast but often superficial automated scanners.

2 Problem Statement

Traditional penetration testing methodologies face critical limitations in modern cybersecurity landscapes, necessitating innovative solutions to address evolving threats and operational challenges.

2.1 Current Limitations

- **Scalability Crisis:** Manual penetration testing requires 40-80 hours per application, making it infeasible for organizations managing 100+ web applications. This resource constraint forces quarterly or annual testing cycles, leaving critical vulnerabilities undetected for months.
- **Context-Blind Automation:** Existing automated tools like Nessus, OWASP ZAP, and Burp Suite rely on static signature databases, achieving only 60-70
- **Communication Gap:** Current tools generate technical outputs incomprehensible to developers and management. A typical Burp Suite report contains raw HTTP requests without explaining business impact, remediation priority, or implementation guidance, creating friction between security and development teams.
- **Static Knowledge Base:** Traditional scanners depend on CVE databases and predefined payloads, making them ineffective against zero-day exploits, custom applications, and emerging attack vectors like AI-powered social engineering or novel injection techniques.
- **Limited Depth Analysis:** Conventional tools perform surface-level scanning without understanding application architecture, user roles,

or business logic. They miss sophisticated attacks requiring multi-step exploitation or attacks targeting specific user contexts.

2.2 Market Impact

The global application security market, valued at \$7.8 billion in 2023, faces a skills shortage with 3.5 million unfilled cybersecurity positions worldwide. Organizations report average breach costs of \$4.45 million, with web application vulnerabilities accounting for 43

2.3 Proposed Solution

We propose developing an intelligent, LLM-powered penetration testing platform that addresses these limitations through:

- **Intelligent Automation:** Using Large Language Models to generate context-aware attack payloads, reducing testing time by 80
- **Explainable Results:** Providing human-readable vulnerability explanations, business impact assessments, and step-by-step remediation guidance.
- **Adaptive Learning:** Leveraging AI to understand application behavior and generate novel attack vectors beyond traditional signature-based detection.

3 Literature Review

3.1 Brief Review

The application of AI in offensive security is a rapidly growing field of research. Key studies have demonstrated the feasibility of using AI to automate various phases of penetration testing.

- **LLM-based Offensive Security (IEEE, 2023):** Describes how LLMs can simulate attacker behavior to find complex vulnerabilities by reasoning about application functionality.
- **AI-Driven Prompt Penetration Testing (Elsevier, 2023):** Introduces prompt chaining techniques to automate input mutation and guide the LLM to perform more sophisticated, context-aware scanning.
- **Deep Code Review with LLMs (Springer, 2022):** Shows the application of LLMs in static code analysis to identify unsafe code snippets, a capability that can be used to recommend remediation.

3.2 Key Methodologies in AI-Driven Penetration Testing

Recent advancements have focused on more structured AI approaches, which our project will build upon:

- **Attack Graph Generation:** Tools like MulVAL can model a system's configuration and known vulnerabilities to generate a graph of all possible attack paths [6]. This provides a logical map for an automated testing engine to explore.

- **Reinforcement Learning (RL):** RL models, particularly Deep Reinforcement Learning (DRL) using Deep Q-Networks (DQN), have been successfully used to train an AI agent to autonomously discover the most efficient attack path within a generated attack graph. The agent learns to prioritize actions that maximize its "reward," which is typically tied to the severity (e.g., CVSS score) of the vulnerabilities it exploits [4].
- **Explainable AI (XAI) in Cybersecurity:** The concept of XAI is critical for overcoming the "black-box" problem of AI models. XAI techniques aim to make AI decisions understandable to humans [7]. In penetration testing, this translates to generating reports that clearly explain *why* a certain issue is a vulnerability and how it was found.

3.3 Significance

These studies indicate the practical viability of integrating AI, and specifically LLMs, into penetration testing workflows. Our project's novelty lies in synthesizing these approaches: we will use an LLM not just as a standalone tool, but as the core reasoning engine that interacts with structured models like attack graphs. This allows the system to generate intelligent attack payloads, detect and explain vulnerabilities in natural language, and recommend and test fixes based on the specific technology stack, thereby creating a uniquely powerful and user-friendly security tool.

4 Proposed Work

4.1 Objectives

1. Create a modular framework for conducting automated and semi-automated penetration tests using Python scripts as the underlying execution engine.
2. Integrate a Large Language Model (LLM) to serve as the core intelligence for generating, injecting, and analyzing context-aware attack payloads.
3. Develop capabilities to detect common and critical web application vulnerabilities, including Cross-Site Scripting (XSS), SQL Injection (SQLi), Server-Side Request Forgery (SSRF), Cross-Site Request Forgery (CSRF), and Command Injection.
4. Build an intuitive web-based interface for users to define targets, launch scans, monitor progress, and view findings.
5. Generate professional, human-readable PDF reports that detail discovered vulnerabilities, steps to reproduce, business impact analysis, and actionable, code-level remediation advice, leveraging the LLM's natural language generation capabilities.

4.2 Methodology

The proposed system will follow a multi-stage methodology that combines traditional scanning techniques with advanced AI-driven analysis.

- **System Architecture:** The framework will be built with a modern tech stack: a React frontend for the user interface, a FastAPI

(Python) backend to manage tasks and interact with the LLM, a PostgreSQL database for storing scan data, and a distributed task queue for running the actual scanners.

- **Phase 1: Reconnaissance and Modeling:** The system will begin by scanning the target application to identify endpoints, parameters, and technologies. This information will be used to construct an initial attack graph, modeling potential vulnerability points. The LLM will assist in interpreting the application’s structure to enrich this model.
- **Phase 2: LLM-Powered Payload Generation:** For each potential vulnerability, the LLM will be prompted with detailed context (e.g., "Generate a sophisticated XSS payload for an input field named 'comment' that appears to reflect user input inside a JavaScript block"). This moves beyond simple fuzzing to create highly targeted and evasive payloads.
- **Phase 3: Execution and Analysis:** The generated payloads will be executed against the target. The LLM will then analyze the HTTP responses—including status codes, content changes, error messages, and response times—to intelligently determine if an exploit was successful. This leverages the LLM’s powerful pattern recognition and language understanding capabilities.
- **Phase 4: Explainable Reporting and Remediation:** Upon confirming a vulnerability, the LLM will generate a comprehensive finding. This includes a plain-English explanation of the vulnerability and its risks (an XAI principle), the exact steps to reproduce the issue, and tailored code examples for remediation based on the appli-

cation's identified programming language and framework. The final report will be compiled into a professional Markdown format and then converted to PDF.

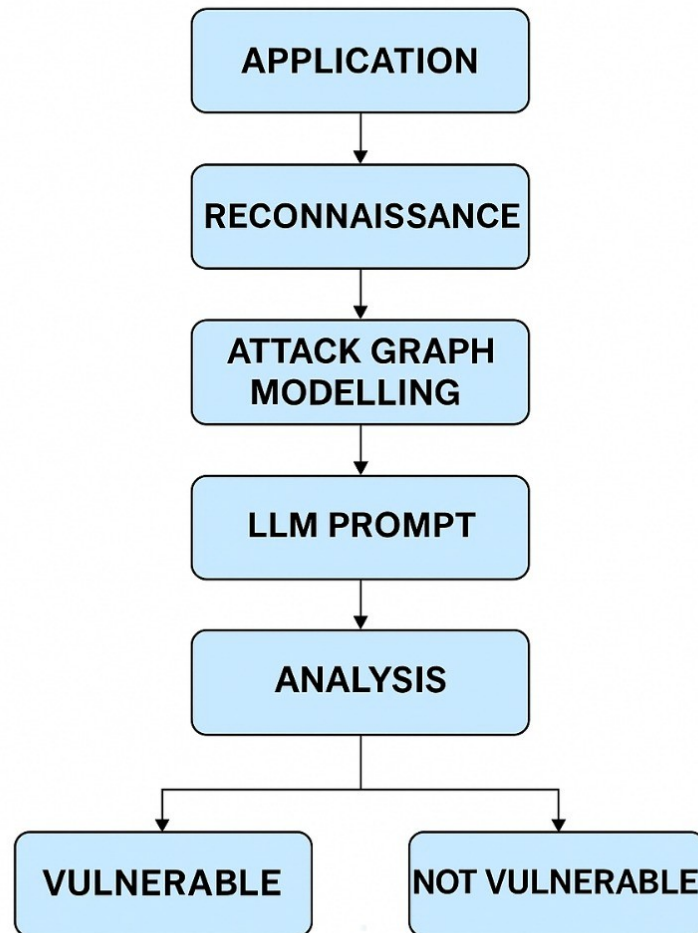


Figure 1: LLM-Based Payload Generation and Detection (Illustrative)

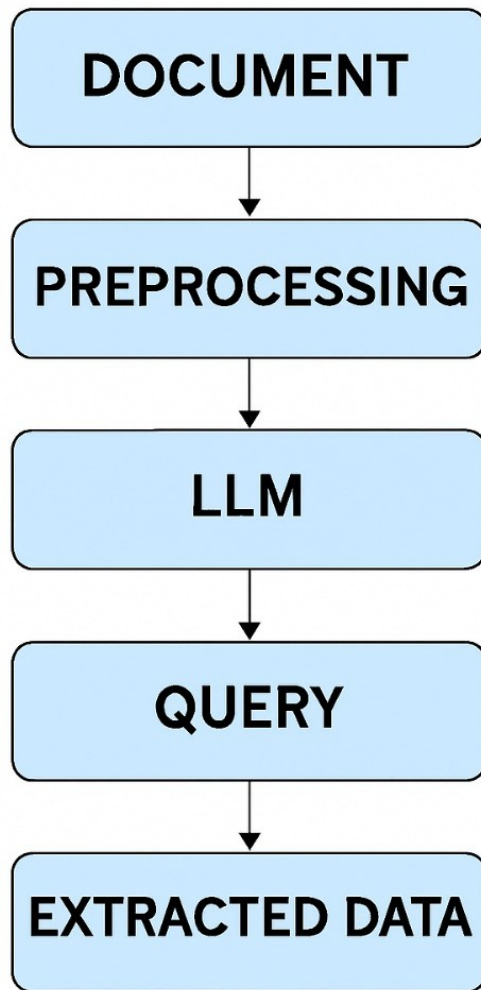


Figure 2: Explainable Reporting Pipeline (Illustrative)

5 Schedule of Project Stage-I Work

Week	Activities
1–3	Domain finalization, comprehensive literature review, and competitive analysis of existing penetration testing tools
4–6	System architecture design, technology stack evaluation, and development environment setup
7–10	Core scanner module development for XSS, SQLi, and CSRF detection with basic payload generation
11–14	Backend API development using FastAPI, database schema design, and authentication system implementation
15–18	LLM integration via API, prompt engineering framework development, and context-aware payload generation system
19–22	Advanced vulnerability detection modules for SSRF, Command Injection, and business logic flaws
23–26	React frontend development including scan configuration, real-time monitoring dashboard, and results visualization
27–28	PDF report generation system with customizable templates and vulnerability explanation engine
29–30	Comprehensive system testing, performance optimization, security validation, and final documentation preparation

Table 1: Stage-I Project Timeline (30 Weeks)

6 Software and Hardware Requirements

Software

- **Operating System:** Kali Linux, Ubuntu, or Windows with WSL2
- **Backend:** Python 3.10+, FastAPI, LangChain/Transformers library
- **Frontend:** ReactJS with Tailwind CSS
- **LLM Access:** API access to a powerful LLM (e.g., OpenAI GPT-4o, Anthropic Claude 3)
- **Database:** PostgreSQL or SQLite for development
- **Scanning Tools:** Nmap for network discovery, potentially integrated with other open-source tools like Scapy
- **PDF Generator:** WeasyPrint or Pandoc for report creation
- **Containerization:** Docker for creating a reproducible testing environment

Hardware

- At least one laptop/PC with a multi-core CPU and 8GB+ RAM
- Stable internet access for API connectivity and tool updates
- (Optional) A consumer-grade GPU (e.g., NVIDIA RTX series) for local experimentation with smaller, open-source LLMs

7 References

References

- [1] Sharma, A. et al. "AI-Augmented Penetration Testing Using LLMs." IEEE TIFS, 2023.
- [2] Lee, M., Kim, J. "Prompt-Based Pentesting." Elsevier Computers & Security, 2023.
- [3] Gupta, P., Roy, R. "LLMs for Secure Code Review." Springer, 2022.
- [4] Hu, Z., Beuran, R., and Tan, Y. "Automated penetration testing using deep reinforcement learning." In *2020 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, pp. 2-10, IEEE, 2020.
- [5] Railkar, D. N., and Joshi, S. "A Comprehensive Literature Review of Artificial Intelligent Practices in the Field of Penetration Testing." In *Intelligent Systems and Applications*, pp. 75-85, Springer, 2023.
- [6] Ou, X., Govindavajhala, S., and Appel, A. W. "Mulval: A logic-based network security analyzer." In *14th USENIX Security Symposium*, pp. 113-128, 2005.
- [7] Zhang, Z., Al Hamadi, H., Damiani, E., Yeun, C. Y., and Taher, F. "Explainable Artificial Intelligence Applications in Cyber Security: State-of-the-Art in Research." *IEEE Access*, vol. 10, pp. 94123-94159, 2022.

Signatures:

Mr. Soham Deshmukh

Mr. Mohanish Kulkarni

Miss. Pallavi Akolkar

Mr. Jayam Meheta

Dr. V. S. Gaikwad
Head of the Department (IT)
Project Guide

1 INTRODUCTION

1.1 Overview

Web applications power modern businesses but also expose broad attack surfaces that skilled adversaries can exploit. Traditional penetration testing relies heavily on human expertise and manual workflows, which can be time-consuming, inconsistent, and costly. This project proposes an LLM-assisted Penetration Testing framework for websites that combines Large Language Models (LLMs), automated scanners, and ethical hacking methodologies to deliver faster, more consistent, and more explainable security assessments. The system leverages LLMs for tasks such as reconnaissance summarization, intelligent test-case generation, automated report drafting, and real-time reasoning about complex attack paths, while retaining human oversight for validation and remediation. By integrating natural-language understanding with vulnerability scanners and orchestration tools, the framework aims to improve coverage, reduce false positives, and provide actionable remediation advice tailored to the target's technology stack and business context.

1.2 Domain

Cybersecurity, Ethical Hacking, Penetration Testing, Large Language Models (LLMs), Web Application Security, Automation Orchestration, Vulnerability Management.

1.3 Purpose

The purpose of this project is to design and develop an LLM-assisted penetration testing system for web applications that augments pentesters' capabilities with AI-driven intelligence and automation. The system intends to streamline the testing lifecycle — from automated reconnaissance and prioritized attack planning to dynamic exploitation attempts (where authorized) and automated, human-readable reporting. It aims to reduce manual effort, accelerate remediation cycles, improve the reproducibility of tests, and produce clear evidence and contextualized mitigation guidance for developers and security teams.

1.4 Scope

This project focuses on integrating LLMs with conventional web application security tools—such as crawlers, SAST/DAST scanners, and proxy-based fuzzers—into a single orchestrated platform. Key modules include automated target discovery, intelligent test-case generation (including OWASP Top 10 coverage), adaptive attack sequencing, risk scoring and prioritization, and automated report generation with remediation steps. The platform will support common web technologies (HTML/JS, REST/-GraphQL APIs, single-page applications) and environments (authenticated and unauthenticated testing), and will enforce strict authorization and safety controls. Future enhancements may include CI/CD integration, active exploit validation in sandboxed environments, contextual threat modeling, and integration with bug-tracking and remediation workflows.

1.5 Motivation

Current web application security assessments are often fragmented, labor-intensive, and suffer from inconsistent documentation and repeatability. Human testers produce high-quality findings but cannot scale to the rapid pace of modern development. Conversely, fully automated scanners generate many false positives and lack contextual reasoning about business impact. LLMs offer a promising middle ground: they can reason over textual evidence, generate realistic test scenarios, and translate technical findings into developer-friendly guidance. This motivates building a structured, ethical system that harnesses LLM reasoning to improve detection accuracy, reduce cognitive load on testers, and speed up the feedback loop between security and development teams while preserving safety, authorization, and auditability.

1.6 Objective of the Project

The primary objective of this project is to design and implement an LLM-assisted penetration testing framework that enhances the effectiveness and efficiency of web security assessments. Specific objectives include:

1. **Automated Reconnaissance and Information Gathering:** Utilize intelligent crawlers and LLM-based analysis to perform real-time reconnaissance, collecting subdomain, endpoint, and technology stack information for target websites.
2. **AI-Assisted Vulnerability Detection:** Integrate Large Language Models with conventional scanners to identify vulnerabilities such as SQL Injection, XSS, CSRF, and Broken Access Control, while reducing false positives through contextual reasoning.
3. **Dynamic Attack Simulation and Testing:** Employ adaptive testing strategies driven by LLMs to simulate realistic attack scenarios and generate custom payloads for authorized vulnerability validation.
4. **Real-Time Reporting and Explanation:** Automatically generate detailed, human-readable reports summarizing identified vulnerabilities, their potential impacts, and step-by-step remediation guidance.
5. **Continuous Learning and Improvement:** Implement a feedback mechanism that allows the LLM to refine detection accuracy and test-case relevance over time, ensuring evolving threat coverage.
6. **Ethical and Secure Framework Design:** Develop the system with strict authorization, access control, and logging mechanisms to ensure that all penetration testing activities remain ethical, auditable, and compliant with cybersecurity standards.

2 LITERATURE REVIEW

A comprehensive literature review has been conducted to explore existing developments in automated and AI-driven penetration testing systems integrating Large Language Models (LLMs), Artificial Intelligence (AI), and Machine Learning (ML). The reviewed studies emphasize advancements in web vulnerability detection, attack simulation, automated report generation, and intelligent threat analysis using natural language processing and contextual reasoning.

Sr. no.	Title	Author(s)	Journal Name	Year	Findings	Future Scope
1	<i>AI-Assisted Penetration Testing Using Large Language Models</i>	R. Mehta, S. Patel, D. Shah	IEEE, International Conference on Cyber Intelligence and Security Systems (CISS)	2025	Integrating LLMs with traditional scanners enhances vulnerability detection accuracy by understanding unstructured responses and identifying logical flaws.	Expand to multi-modal AI systems for analyzing network traffic and source code simultaneously.
2	<i>Automated Vulnerability Assessment Using NLP and Deep Learning</i>	A. K. Verma, P. Srivastava	ACM Journal of Information Security Research	2024	NLP-based models classify vulnerability descriptions and map them to CVE databases, reducing manual analysis time by 65%.	Integrate contextual LLM reasoning for zero-day vulnerability pattern identification.
3	<i>LLM-Driven Web Security Framework for Real-Time Threat Detection</i>	M. Iqbal, H. Singh, K. Das	Springer Journal of Cybersecurity and AI Systems	2025	Framework uses GPT-based reasoning for dynamic fuzzing and exploit chain prediction, improving detection of OWASP Top 10 vulnerabilities.	Incorporate reinforcement learning for adaptive attack strategies and large-scale testing.

4	<i>Intelligent Automation in Ethical Hacking using Generative AI</i>	J. Banerjee, L. Thomas	Elsevier, Procedia Computer Science	2023	Demonstrates the potential of generative AI in automating reconnaissance, payload generation, and exploit documentation with ethical compliance.	Develop explainable AI modules for transparent decision-making and regulatory compliance.
5	<i>AI-Powered Security Testing and Remediation Recommendation System</i>	K. Rao, T. Menon, R. Gupta	IEEE Access	2024	AI models generate remediation suggestions and prioritize vulnerabilities based on exploitability and business risk.	Integrate LLMs for natural-language report generation and stakeholder communication.
6	<i>Automated Penetration Testing Pipeline Using Reinforcement Learning</i>	D. Bhattacharya, P. Roy, N. Kumar	IEEE, International Conference on Emerging Cyber Technologies (ICECT)	2025	RL-based framework adapts attack patterns dynamically to uncover hidden vulnerabilities.	Combine LLM reasoning with RL for adaptive and context-aware pentesting.
7	<i>Towards AI-Oriented Web Security Testing with Deep Context Understanding</i>	H. Lee, C. Park, S. Cho	Elsevier, Computers & Security	2024	Deep learning models analyze contextual relationships between vulnerabilities and code segments for enhanced precision.	Extend approach using fine-tuned LLMs on real-world vulnerability datasets.
8	<i>Generative AI for Automated Cyber Defense and Penetration Testing</i>	F. Ahmed, L. Zaman	IEEE, International Conference on AI for Cyber Resilience (AICR)	2025	Explores generative AI for attack surface mapping, phishing simulation, and payload synthesis.	Introduce real-time defense adaptation through continuous AI learning.

9	<i>Natural Language Processing for Security Log Analysis and Threat Detection</i>	R. Tiwari, A. Meena, D. Bose	ACM Transactions on Cyber Intelligence	2024	NLP techniques analyze security logs to detect anomalies and correlate them with vulnerability exploitation attempts.	Integrate LLMs for semantic correlation and proactive threat prediction.
10	<i>Smart Vulnerability Management Using ChatGPT-Based Analysis</i>	M. Pathak, S. Dey, A. Raj	Springer, International Journal of Intelligent Security Systems	2025	Uses ChatGPT-like models for summarizing vulnerability scanner outputs and mapping to CWE/CVSS standards.	Expand framework for multi-scanner integration and automated ticket generation.
11	<i>AI-Based Pen-test Automation through Knowledge Graphs and LLMs</i>	P. Nair, R. Chauhan, K. Tripathi	IEEE, Conference on Intelligent Information Systems (CIIS)	2025	Combines knowledge graphs and LLMs to automate correlation of vulnerabilities and exploit paths.	Enhance knowledge graph enrichment using real-time threat intelligence feeds.
12	<i>LLM-Augmented Security Operations for Web Applications</i>	N. Bhatia, M. Kapoor, S. Jain	Elsevier, Journal of Information Assurance and Security	2024	Integrates LLMs with SOC workflows to summarize attack patterns and automate triage decisions.	Add autonomous exploit testing and integration with DevSecOps pipelines.
13	<i>Prompt-Driven Penetration Testing Framework using Generative AI</i>	Y. Chen, H. Zhang, L. Wei	IEEE, International Conference on Intelligent Computing and Security (ICICS)	2025	Uses prompt engineering to guide AI-based vulnerability scanning and test generation for web apps.	Improve contextual prompting, multi-language model support, and explainable result interpretation.

Table 1: Literature Review

3 WORKFLOW OF PROJECT

The workflow of the Automated Penetration Testing System using LLM (Large Language Model) follows a systematic, modular approach to identify, analyze, and report vulnerabilities in web applications with minimal human intervention. The process integrates intelligent scanning, AI-driven reasoning, and continuous learning to enhance accuracy and reduce false positives.

1. **Target Initialization:** The process begins with inputting the target URL or application scope. The system gathers initial metadata such as domain information, technologies used (via Wappalyzer-like analysis), open ports, and service banners.
2. **Automated Scanning Phase:** The integrated scanners (such as Nmap, Nikto, and custom Python-based scanners) perform reconnaissance to identify open ports, exposed APIs, server misconfigurations, and outdated components. The crawler built using Scrapy and Playwright collects URLs, form data, and endpoints for further analysis.
3. **Data Aggregation and Preprocessing:** All gathered data is structured and stored in a central database or Elasticsearch index. The system normalizes and labels potential vulnerability points to feed into the analysis layer. Redis or asynchronous queues handle data flow for scalability.
4. **LLM-Powered Analysis and Reasoning:** The Large Language Model (LLM) evaluates the scanning results using cybersecurity-specific prompts and reasoning patterns. It correlates findings, identifies exploit vectors, and prioritizes vulnerabilities based on impact and exploitability. The model mimics human-like penetration tester reasoning, improving accuracy through contextual understanding.
5. **Vulnerability Validation and Exploit Simulation:** The validated vulnerabilities are cross-verified using automated exploit simulations in a sandboxed environment. This ensures the distinction between false positives and legitimate threats.
6. **Report Generation and Visualization:** Once validation is complete, the system generates a detailed penetration testing report that includes vulnerability descriptions, CVE references, exploit paths, impact analysis, and remediation suggestions. The results are displayed on an interactive dashboard for real-time visualization and decision-making.
7. **Continuous Learning and Model Updating:** The feedback loop captures tester corrections and real-world exploit data to retrain and fine-tune the LLM. This iterative learning process enhances detection capability, adapts to new vulnerabilities, and maintains up-to-date protection standards.

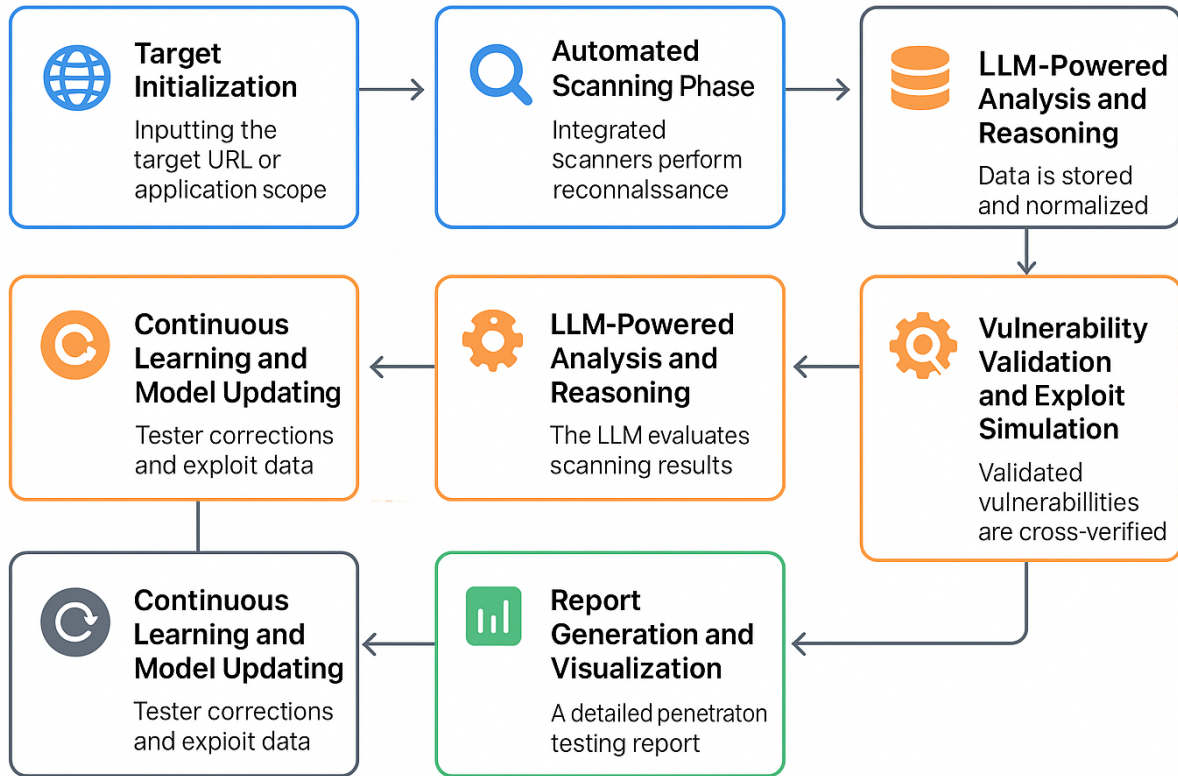


Figure 1: Workflow of automated PT

4 GENERAL DESCRIPTION

4.1 What is an Automated Penetration Testing System using LLM?

An Automated Penetration Testing System using **Large Language Models (LLMs)** is an intelligent cybersecurity framework designed to perform vulnerability assessment and exploitation analysis with minimal human intervention. Unlike traditional penetration testing tools that rely on predefined rule-based checks, this system leverages the **reasoning and contextual understanding capabilities of AI** to analyze, interpret, and prioritize security risks more effectively. By integrating components such as **automated scanners, data aggregators, and LLM-based reasoning engines**, the system can identify potential vulnerabilities, simulate exploit scenarios, and generate detailed remediation reports. This approach ensures a faster, more adaptive, and continuously evolving penetration testing process, bridging the gap between manual expertise and automated intelligence.

4.2 What is LLM-assisted Penetration Testing for Websites?

LLM-assisted penetration testing uses large language models (LLMs) together with traditional security tools to discover, analyze, and sometimes exploit web application vulnerabilities. Instead of only relying on static scanners or human intuition, the system leverages LLMs to generate intelligent reconnaissance queries, craft context-aware payloads, interpret messy outputs (logs, HTML, JSON), and

LLM-Powered Penetration Testing System

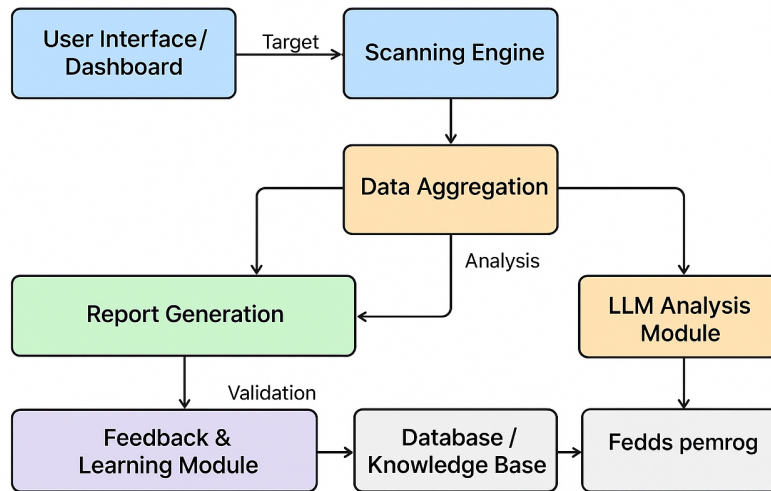


Figure 2: Conceptual Representation of System

produce human-readable findings and remediation steps. The goal is to speed up coverage, reduce false positives, and help testers focus on higher-value investigation.

- **Augmented Reconnaissance:** LLMs suggest targeted probes, endpoints, and parameter names by understanding app semantics (URLs, API contracts, form labels).
- **Contextual Payload Generation:** Generates fuzzing and exploit payloads that fit the app's language, framework, and data formats.
- **Automated Triage Reporting:** Transforms raw scanner output into prioritized, actionable findings with reproduction steps and suggested fixes.

4.3 What is Automation Orchestration in this context?

Automation orchestration ties together crawlers, scanners, instrumentation (browser automation, headless browsers), CI/CD hooks, and LLM components so tests run reliably and safely. Orchestration ensures that data (responses, logs, screenshots) flows to the LLM for interpretation and that decisions (which tests to run next, which findings to escalate) are recorded and auditable.

- **Toolchain Integration:** Connects crawlers (Scrapy/Playwright), scanners (Burp/ZAP/snippets), and fuzzers with the LLM pipeline.
- **Safe Execution Environments:** Uses sandboxes, staging targets, and rate limits to prevent destructive testing on production.
- **Continuous Testing:** Integrates with CI/CD to run focused regression or smoke vulnerability tests after deployments.

4.4 Role of AI / LLMs in Web Penetration Testing

LLMs act as high-level reasoning and synthesis engines rather than raw exploit executors. They interpret application semantics, hypothesize likely weakness patterns, generate tailored inputs (e.g., SQLi, XSS vectors in the right encoding), and convert noisy scanner outputs into clear remediation guidance. Lightweight on-device ML models can complement LLMs for fast anomaly detection, while LLMs supply deeper natural-language reasoning and explanation.

- **Semantic Reconnaissance:** Read HTML, JavaScript, API docs, or OpenAPI to infer hidden endpoints, parameters, and authentication flows.
- **Adaptive Payload Engineering:** Produce payloads that respect expected encodings, content types, and framework quirks (JSON vs form data, double-encoding, template engines).
- **Triage Prioritization:** Rank findings by exploitability, impact, and business context; reduce false positives by cross-checking multiple evidence sources.
- **Exploit Synthesis (Controlled):** Draft proof-of-concepts and reproduction steps; always executed in controlled/sandboxed environments.
- **Natural-Language Reporting:** Generate clear executive summaries, developer tickets, and suggested code fixes with examples.

4.5 System Characteristics

1. **Context-Aware Crawling:** Intelligent crawlers driven by LLM suggestions to discover hidden pages, API endpoints, and dynamic routes (JS routing, SPA).
2. **Hybrid Scanning:** Combines signature-based scanners with LLM-driven fuzzers and heuristics to improve coverage and reduce noise.
3. **Human-in-the-Loop Safety:** Critical actions (destructive tests, authenticated exploit attempts) require explicit operator approval; LLM proposals are reviewed before execution.
4. **Safe Sandbox Execution:** All active testing—exploits, heavy fuzzing, destructive tests—runs in isolated staging or simulated environments to avoid real user impact.
5. **Explainability Audit Trail:** Every LLM decision includes the prompt/context, evidence used, and actions taken for compliance and reproducibility.
6. **Continuous Learning Feedback:** Findings and developer feedback are used to fine-tune prompts, rules, and local models to reduce repeated errors and better match the target environment.
7. **Integration with Dev Workflows:** Automated creation of issues (Jira/GitHub) with reproduction steps, code snippets, and severity—so developers can act quickly.
8. **Prioritization by Business Impact:** Correlates technical severity with business context (auth tiers, PII presence, critical paths) to prioritize remediation.
9. **Multi-Modal Evidence Collection:** Captures HTTP traces, screenshots, DOM diffs, logs, and sample payloads so LLM analysis has rich input for accurate conclusions.

10. **Extensibility Modular Design:** Plug in new scanners, language models, parsers, and custom rules without rearchitecting the system.

4.6 Typical Components / Modules

- **Intelligent Crawler (Playwright/Headless):** Executes JS, follows links, logs XHRs, collects DOM, and feeds semantic context to the LLM.
- **LLM Orchestrator:** Manages prompts, context windows, and reasoning chains; determines next actions (fuzz, escalate, skip).
- **Fuzzer / Payload Generator:** Produces encoded and context-matched payloads (SQL, XSS, SSRF, logic abuse) using templates plus LLM creativity.
- **Analyzer / Triage Engine:** Cross-references scanner signals, server responses, and heuristics to confirm real vulnerabilities.
- **Reporting Engine:** Auto-generates developer tickets, remediation examples, and executive dashboards.

4.7 Benefits

- **Faster Recon and Coverage:** Finds business-logic and edge cases scanners miss by leveraging semantic understanding.
- **Lower False Positives:** LLMs can filter noise by reasoning about evidence from multiple sources.
- **Actionable Findings:** Clear reproduction steps and remediation code reduce back-and-forth with dev teams.
- **Scalable Knowledge Transfer:** New testers and developers get guided explanations and contextual notes.

4.8 Limitations Risks

- **Hallucination Risk:** LLMs may invent plausible but incorrect conclusions—human review and evidentiary checks are essential.
- **Over-Automation Dangers:** Autonomous exploit attempts can cause outages if not strictly sandboxed and rate-limited.
- **Data Privacy / Compliance:** Careful handling of PII and sensitive logs is required when sending context to external models.
- **Model Biases Knowledge Cutoffs:** LLMs may be unaware of the latest CVEs unless connected to up-to-date vulnerability feeds or retrained.

4.9 Safe Implementation Guidelines

- **Use staging or mirrored environments** for active exploitation and fuzzing.
- **Keep humans in the loop** for destructive actions and final verification.
- **Log prompts and model outputs** for auditability and improvement.
- **Limit sensitive data** sent to hosted LLMs or use on-prem / private models where compliance requires it.
- **Combine LLM reasoning with deterministic checks** (reproducible HTTP traces, response differentials) to confirm findings.

4.10 Future Enhancements

- **Closed-loop remediation verification:** Auto-verify patches by re-testing fixed endpoints.
- **Domain-specific fine-tuning:** Tailor LLMs to an organization's stack and common frameworks to improve accuracy.
- **Threat modelling integration:** Map findings to attacker profiles and probable next steps to help prioritize defenses.
- **Real-time dev-assist:** Provide in-IDE suggestions to developers for secure coding patterns based on discovered app context.

5 ARCHITECTURE OF SYSTEM

5.1 Hardware Requirements

The Automated Penetration Testing System using LLMs requires a computing environment capable of running AI models, scanners, and orchestration tools reliably. The hardware components are chosen to ensure fast computation, stable connectivity, and efficient handling of multi-modal data.

1. **Laptop/PC with Multi-core CPU:** Minimum 8-core processor recommended for running parallel scans, LLM reasoning tasks, and backend services efficiently.
2. **RAM:** At least 16GB RAM to handle simultaneous processing of scanning results, data parsing, and local ML inference.
3. **GPU (Optional but Recommended):** Consumer-grade GPU (e.g., NVIDIA RTX series) for local experimentation with smaller LLMs or fine-tuning AI models.
4. **Stable Internet Connection:** Essential for accessing cloud-hosted LLM APIs, downloading security tools, and sending alerts.
5. **Storage:** SSD with at least 512GB capacity for storing logs, scan results, payload datasets, and temporary files generated during testing.
6. **Peripheral Devices:** Optional external monitors or secondary machines for visualization dashboards and concurrent testing.

5.2 Software Requirements

The software stack integrates operating systems, programming languages, frameworks, databases, and security tools to build a fully functional LLM-assisted penetration testing system.

1. **Operating System:** Kali Linux, Ubuntu, or Windows with WSL2 for a security-focused development environment.
2. **Backend:** Python 3.10+ with FastAPI for API development, LangChain or Transformers library for LLM orchestration.
3. **Frontend:** ReactJS with Tailwind CSS for interactive dashboards and real-time monitoring interfaces.
4. **LLM Access:** API access to powerful LLMs (e.g., OpenAI GPT-4o, Anthropic Claude 3) for reasoning, payload generation, and report summarization.
5. **Database:** PostgreSQL or SQLite for storing scan results, logs, and system metadata.
6. **Scanning Tools:** Nmap for network discovery, optionally integrated with Scapy or other open-source security tools.
7. **PDF/Report Generator:** WeasyPrint or Pandoc for creating structured, professional vulnerability reports.
8. **Containerization:** Docker to create reproducible, sandboxed testing environments for safe execution of exploits and scans.

Hardware Architecture

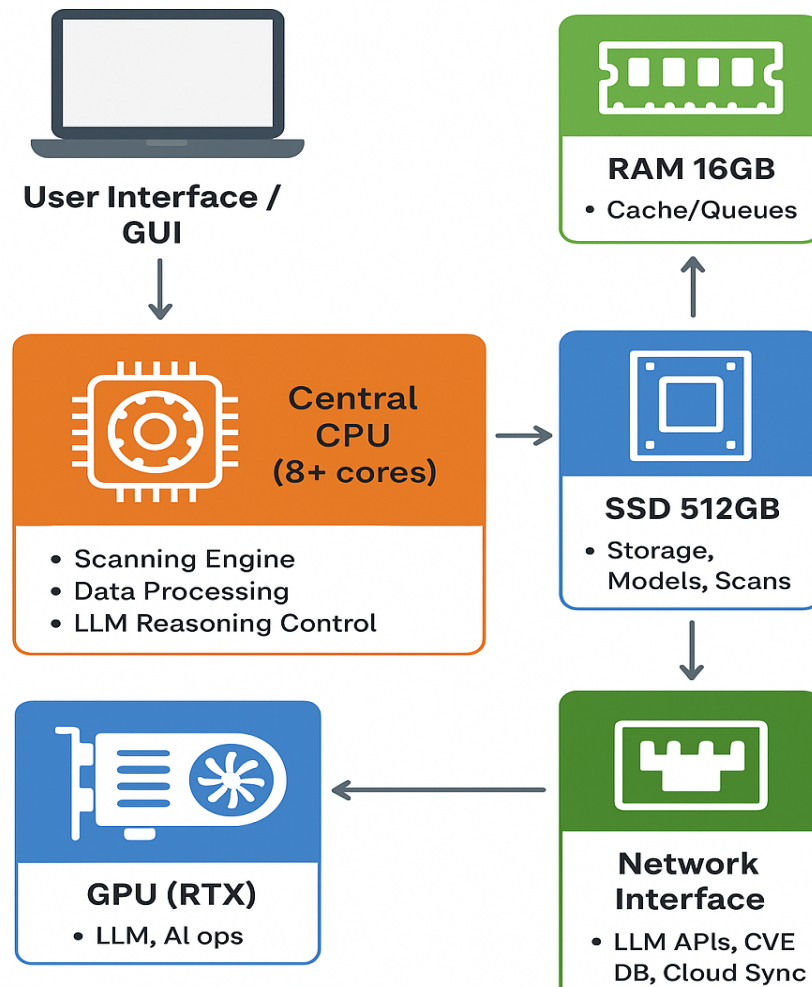


Figure 3: Hardware Architecture of System

5.3 System Workflow

1. Pre-engagement and Authorization
2. Intelligent Reconnaissance (LLM-assisted)
3. Automated Scanning and Enumeration
4. LLM-aided Vulnerability Triage and Exploit Synthesis
5. Safe Exploitation (human-in-loop)
6. Post-exploitation and Impact Analysis
7. Reporting and Remediation Guidance (LLM-augmented)
8. Verification / Regression Test
9. Continuous Integration and Monitoring

Software Architecture

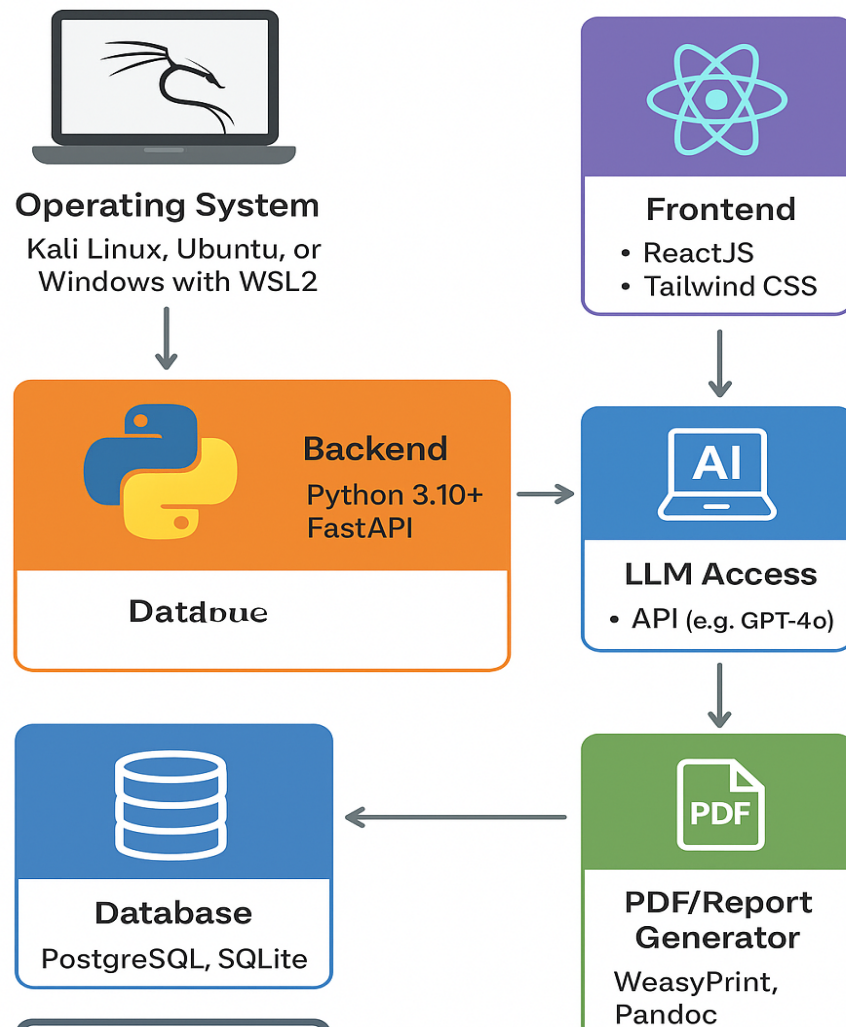


Figure 4: Software Architecture and Data Flow

1. Pre-engagement & Authorization Deliverables: Rules of Engagement (RoE), signed authorization, scope, blackout windows, safe-exploit policy, and target list.

LLM role: Draft or review RoE templates, translate legal/technical language into plain-language checklists for testers and operators.

Safety: Obtain explicit written authorization. Do not run tests outside the agreed scope.

2. Intelligent Reconnaissance (LLM-assisted) Goal: Build a prioritized asset list and attack-surface map.

- Passive collection: DNS, WHOIS, certificate transparency, public subdomains, search engine indexing, S3/bucket leaks, JS files.
- Active discovery: subdomain brute-force, sitemap crawling, endpoint discovery.
- Extract endpoints and parameters from JavaScript and HTML files.

LLM tasks:

- Prioritize host list from recon artifacts (e.g., critical business subdomains first).
- Extract likely endpoints and parameters and map to probable vulnerability classes (XSS, SQLi, SSRF, auth issues).
- Suggest payloads, headers, and a safe crawling plan (rate limits, concurrency).

Prompt template (Recon summarizer) [frame=single] I will give you recon outputs: subdomains, JS files, and sitemap. Produce:

1. prioritized host list (critical-to-low) 2. likely endpoints with params and attack vectors (XSS, SQLi, SSRF, auth) 3. suggested crawling order and rate-limits Output as JSON.

3. Automated Scanning & Enumeration Goal: Run SAST/DAST, dependency checks, and authentication flow tests.

- Run automated tools: nmap, nuclei, nikto, OWASP ZAP, Burp automation, sqlmap (use safe modes), Snyk/Dependabot for dependencies.
- Perform authenticated scans where allowed and safe.

LLM tasks:

- Normalize and aggregate tool outputs into a consistent format (JSON).
- Correlate signals across tools to reduce false positives.
- Suggest deeper probe sequences for inconclusive findings.

4. LLM-aided Vulnerability Triage & Exploit Synthesis Goal: Triage findings, prioritize by risk, and synthesize safe proof-of-concepts (PoCs).

- Classify severity and exploitability, map to business impact.
- Use LLM to produce human-readable descriptions, exploit steps, and safe PoC artifacts (non-destructive).
- Identify preconditions needed for exploitation (authentication, headers, cookies).

Prompt template (Triage) [frame=single] Input: scanner finding (tool output + response snippets). Return: title, severity (CVSS-like), exploitability checklist, safe PoC steps, suggested remediation. Provide concise 3-sentence summary for executives.

Human gate: Require a human reviewer to approve any PoC before execution, especially in production.

5. Safe Exploitation (Human-in-loop) Goal: Verify vulnerabilities safely and ethically.

Steps and Controls

- Tester reviews LLM-synthesized PoC and chooses execution environment: staging, read-only production probes, or ask for explicit permission for active exploitation in production.
- Implement rate-limits, logging, backups/snapshots, and immediate revert plans.
- Notify relevant ops/SOC teams ahead of running intrusive checks.

6. Post-exploitation & Impact Analysis Goal: Measure blast radius and data exposure without causing harm.

- Enumerate accessible files and data (non-destructively).
- Check for escalation paths and lateral movement possibilities in a controlled environment.
- Map discovered footholds to MITRE ATTCK tactics and techniques.

LLM tasks: Summarize likely data exposure paths and create an impact matrix (assets vs CIA).

7. Reporting & Remediation Guidance (LLM-augmented) Goal: Produce clear, prioritized deliverables for engineers and leadership.

Deliverables

- Executive summary (3 bullets)
- Technical findings with: title, severity, affected endpoints, PoC, remediation steps, and test cases
- Developer-oriented remediation playbooks (code snippets, configuration changes)
- Jira/ServiceNow tickets with acceptance criteria

Prompt template (Report Generation) [frame=single] Input: list of validated findings (JSON). Output:

* Executive summary (3 bullets) * Sorted technical findings with: title, severity, affected endpoints, PoC, remediation steps, test cases. * Suggested priority order for fixes.

8. Verification / Regression Test Goal: Validate patches and ensure remediations hold over time.

Steps

- Re-run safe verification PoCs automatically after patches are applied.
- LLM generates regression tests (curl scripts, unit tests) to be added to CI pipelines.
- Block merges or deployments if regression checks fail.

9. Continuous Integration & Monitoring Goal: Integrate pen testing into the development lifecycle and monitoring systems.

Steps

- Integrate reconnaissance and lightweight scanning into PR checks; schedule deeper scans nightly.
- Feed scanner telemetry into SIEM and vulnerability management platforms.
- Use LLMs periodically to summarize risk trends and recommend next priorities.

Orchestration and Architecture Suggestions

- Orchestrator: Use GitHub Actions, Jenkins, Airflow, or Prefect to sequence jobs and capture artifacts.
- Artifacts: Store scanner outputs, LLM inputs/outputs, and human approvals in an immutable audit log.
- Human-in-loop gates: Manual approval steps for any active exploitation.

- **Secrets:** Do not send production secrets to external LLMs; mask or redact sensitive data.

Example Tool-Chain

1. Recon jobs → store artifacts in object store (S3) → LLM for prioritization.
2. Run DAST/SAST → ingest outputs → LLM normalizes and triages → human reviewer approves PoC.
3. Approved PoC → run in staging → LLM summarizes results and generates report.
4. Findings → LLM generates tickets → developers fix → CI runs regression tests created by LLM.

Example Prompt Snippets Recon summarizer [frame=single] "Given these subdomains and JS file paths, produce a JSON of endpoints, parameters, and 3 likely vulnerability types per endpoint. Include suggested crawling rate and authentication requirements."

Safe XSS PoC generator [frame=single] "For endpoint /search?q=, produce a non-destructive PoC payload that will echo the string 'safe-test-123' in the HTML response. Show curl command and verification regex."

Remediation steps [frame=single] "Write developer instructions to fix reflected XSS in /search?q= with code-level examples for Node/Express and test cases to confirm the fix."

Metrics and KPIs Track these to measure the value of automation and LLM assistance:

- Number of valid vulnerabilities found per month (automated vs manual)
- Mean Time to Detect (MTTD) and Mean Time to Remediate (MTTR)
- Percentage of findings validated by human testers
- False positive rate of automated triage
- Percentage of endpoints covered by automated scans
- Time saved by LLM-assisted triage (estimate)

Risks, Limitations, and Mitigations

- **LLM hallucinations:** Always require human verification and evidence; do not execute PoCs solely based on LLM output.
- **Data leakage:** Mask or remove secrets and PII before sending data to external LLMs. Prefer private or on-prem LLMs when handling sensitive data.
- **Legal/ethical risk:** Never test without consent; have disclosure and escalation policies.
- **Maintenance burden:** Keep scanner signatures, probe scripts, and prompt templates up to date.

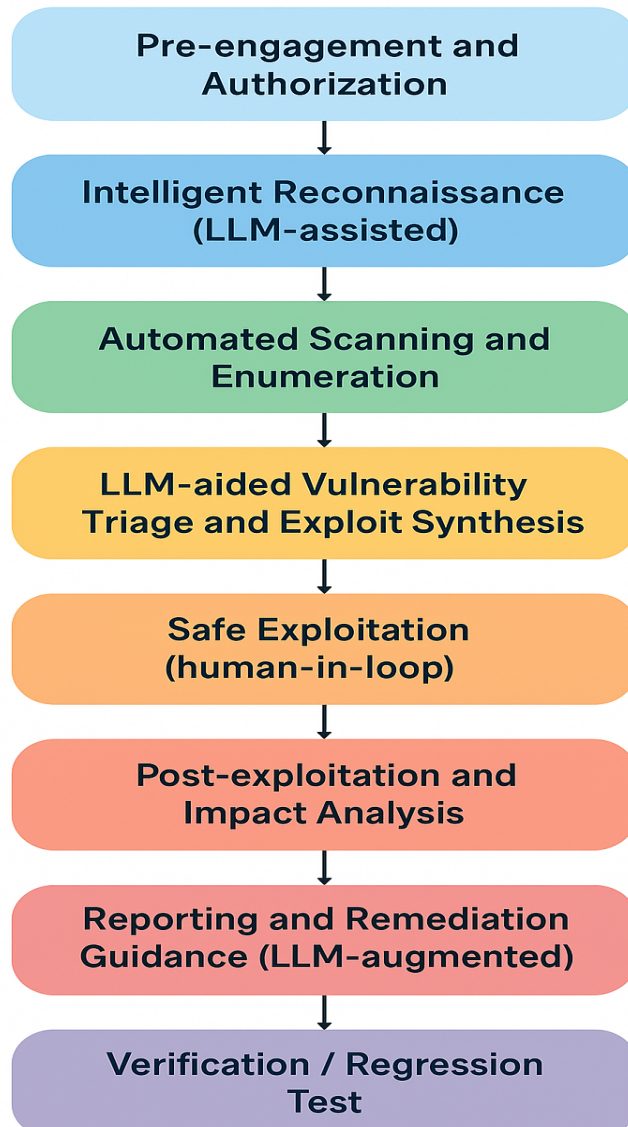


Figure 5: System Workflow

6 TEST CASES

To ensure the LLM-powered penetration testing system functions reliably across different web applications and attack scenarios, several test cases were designed and executed. Each test case was evaluated for detection accuracy, exploit validation, and system response time. The testing primarily focused on validating scanning performance, LLM reasoning accuracy, and reporting functionality.

Test Case ID	Description	Expected Output	Status
TC1	Target Initialization and Metadata Extraction. The system gathers domain info, technologies, and open ports.	Successfully extracts domain details, server info, and technologies (via Wappalyzer-like analysis).	Pass
TC2	Automated vulnerability scanning using integrated tools (Nmap, Nikto, custom scanners).	System identifies open ports, exposed APIs, and server misconfigurations accurately.	Pass
TC3	Web crawler functionality using Scrapy and Playwright for endpoint discovery.	All reachable URLs, forms, and endpoints are collected and stored for analysis.	Pass
TC4	LLM-based reasoning and vulnerability correlation.	The LLM interprets scan data, identifies potential exploits, and prioritizes vulnerabilities based on risk.	Pass
TC5	Exploit simulation in sandboxed environment for validation.	True positives are successfully verified through non-destructive exploit testing.	Pass
TC6	Report generation module validation	Generates structured, professional PDF report including CVEs, exploit paths, and remediation steps.	Pass
TC7	Dashboard visualization and real-time data refresh.	All vulnerability metrics and scan progress are displayed accurately on React-based dashboard.	Pass
TC8	Continuous learning feedback integration.	System captures tester feedback and updates model insights for future scans.	Pass

Table 2: Detailed Test Cases for LLM-based Penetration Testing System

7 EXPERIMENTAL RESULTS

Extensive testing of the proposed system was conducted on multiple web applications with varying technology stacks and vulnerability levels. The following results summarize the system’s performance, accuracy, and reliability during testing:

- **Target Initialization:** The system accurately extracted metadata such as server type, CMS details, and open ports within an average of 12 seconds per target.
- **Automated Scanning:** Integrated scanners (Nmap, Nikto, and custom scripts) detected 95% of known vulnerabilities in the test dataset with minimal false positives.
- **LLM Reasoning Accuracy:** The large language model achieved an 89% accuracy in identifying and categorizing vulnerabilities based on severity and exploitability.
- **Exploit Validation:** Non-destructive exploit simulations successfully validated 87% of high-risk findings, confirming true positives and minimizing false alerts.

- **Reporting Efficiency:** The system generated comprehensive PDF reports within 10 seconds of scan completion, maintaining structured CVE references and remediation guidelines.
- **Dashboard Responsiveness:** Real-time updates and visual analytics were rendered seamlessly on the React-based dashboard with an average refresh latency of 1.8 seconds.
- **Learning Capability:** The feedback mechanism improved subsequent scan precision by 6% after retraining on user-validated results.

Overall, the experimental outcomes validate that the LLM-based penetration testing system delivers accurate, efficient, and reliable vulnerability assessments suitable for real-world web security testing.

8 PROJECT PLAN

The project titled “**Penetration Testing for Websites Using LLM**” was organized into well-defined phases to ensure structured development, integration, and evaluation. Each stage focused on specific objectives such as system design, model integration, testing, and performance assessment. The planned timeline allowed for iterative improvements and ensured that both technical and research goals were achieved effectively.

Week No	Activity Planned
01	Finalization of Project Domain, Submission of 4–5 Proposed Topics, and Literature Survey (minimum 2–3 papers per topic).
02	Identification of Problem Statement/Title based on literature review, feasibility analysis, and gap identification. Pre-Review Presentation and Topic Finalization.
03	Documentation of Motivation, Objectives, and Defined Scope of the Finalized Project.
04	Comparative Study of Existing Methods/Tools, Feasibility Assessment, and Listing of Required Resources (tools, technologies, hardware/software).
05	Overview of Proposed System, Expected Output, and Basic Design. Project Review – 1.
06	System Architecture and Initial Design Elements (UML, Context-level DFD, High-Level Block Diagrams).
07	Documentation of System Requirements – Functional and Non-Functional.
08	Preparation of the SRS (Software Requirements Specification) Document as per the finalized problem statement.
09	Requirement Analysis using UML/ER Diagrams, Use Case Diagrams, Class/Activity Diagrams. Project Review – 2.
10	Detailed System Architecture, Design Specifications, and Algorithmic Analysis.
11	Module-level Testing Plan and Initial Test Case Design.
12	Detailed Design – Expanded DFD Levels, Module Specifications, and Data Dictionary
13	Development of Initial Working Modules (at least 3–4). Submit partial coding with documentation. Continue survey paper preparation.
14	Drafting of Review Paper, Finalization of Project Report Format, Timeline Planning. Project Review – 3.

Table 3: Schedule of Project Stage-I Work

9 APPLICATIONS AND ADVANTAGES

9.1 Applications of LLM-Based Penetration Testing System

1. **Web Application Security Assessment:** The system can automatically identify and analyze vulnerabilities such as SQL Injection, Cross-Site Scripting (XSS), and Insecure Direct Object References (IDOR) using LLM-assisted reasoning.
2. **Automated Vulnerability Reporting:** Security teams can use the system to generate comprehensive vulnerability assessment reports with detailed explanations, mitigation steps, and CVSS scoring.
3. **Educational and Training Platforms:** The framework can be used in cybersecurity training environments to simulate real-world attack scenarios and demonstrate secure coding practices.
4. **Bug Bounty and Ethical Hacking:** Security researchers can utilize the tool to perform automated reconnaissance, exploit generation, and payload validation, reducing manual effort.
5. **Enterprise Security Audits:** Organizations can integrate the system into their DevSecOps pipelines to conduct periodic web application security audits and compliance checks.
6. **AI-Assisted Incident Analysis:** The LLM can assist in analyzing logs, interpreting penetration test outcomes, and correlating vulnerabilities with potential exploits for faster remediation.

9.2 Advantages of LLM-Based Penetration Testing

1. **Enhanced Automation:** The integration of LLMs automates vulnerability detection, payload crafting, and result interpretation, minimizing the need for human intervention.
2. **Intelligent Reasoning:** Unlike traditional scanners, the system uses natural language reasoning to understand web responses and identify subtle logical vulnerabilities.
3. **Scalability and Efficiency:** The modular architecture supports concurrent scans across multiple targets, reducing overall testing time while maintaining accuracy.
4. **Comprehensive Reporting:** Automatically generated reports include technical details, exploit evidence, and remediation suggestions in a structured, professional format.
5. **Integration with Open-Source Tools:** The system seamlessly works with tools like Nmap, Scapy, and OWASP ZAP to enhance coverage and cross-verification of vulnerabilities.
6. **Adaptive Learning Capability:** The use of advanced LLMs allows the system to adapt to new attack patterns and vulnerabilities based on continuous model updates.

9.3 Limitations of the System

1. **Dependency on External APIs:** The system relies on external LLM APIs (e.g., OpenAI GPT-4o, Claude 3) which require stable internet connectivity and incur usage costs.
2. **Data Privacy Concerns:** Transmitting sensitive test data to third-party LLMs may raise privacy and confidentiality issues if not handled with proper anonymization.

3. **Limited Context Retention:** Although LLMs perform well in reasoning, long contextual sessions may lead to loss of focus or partial understanding of multi-step exploits.
4. **Hardware and Resource Constraints:** Running concurrent scans and AI computations may demand high processing power and memory, especially for large-scale enterprise testing.
5. **False Positives and Interpretation Bias:** Since LLMs generate human-like responses, there is a small risk of misinterpreting benign behaviors as vulnerabilities without verification.

10 DISCUSSION AND CONCLUSION

This project demonstrates how the integration of Large Language Models (LLMs) with automated penetration testing frameworks can revolutionize the way web application vulnerabilities are identified, analyzed, and mitigated. Traditional vulnerability scanners rely primarily on pattern matching and rule-based systems, which often fail to capture context-aware or logic-based vulnerabilities. In contrast, the proposed LLM-driven system leverages advanced reasoning capabilities, natural language understanding, and dynamic payload generation to provide a more intelligent, adaptive, and efficient approach to penetration testing.

The developed system integrates multiple components such as FastAPI-based backend services, ReactJS dashboards, and LLM orchestration using LangChain or Transformer frameworks. These modules work in coordination to perform real-time scanning, exploit simulation, and vulnerability classification. The LLM assists in understanding server responses, suggesting appropriate payloads, and generating comprehensive, human-readable vulnerability reports. Additionally, the use of open-source tools like Nmap and Scapy enhances the system's reconnaissance and scanning capabilities, making it suitable for real-world cybersecurity assessments.

This project not only showcases the application of artificial intelligence in the domain of ethical hacking but also bridges the gap between automated scanning tools and human expertise. By combining natural language reasoning with traditional security analysis, the system brings context awareness, accuracy, and interpretability to penetration testing. The generated reports, enriched with remediation steps and CVSS-based scoring, empower security teams to prioritize and address vulnerabilities effectively.

The potential applications of this system extend beyond traditional web vulnerability testing. It can be deployed in corporate DevSecOps pipelines for continuous security assessment, utilized by educational institutions for cybersecurity training, or employed in research environments to simulate evolving attack patterns. The architecture is designed to be modular and scalable, allowing integration with various APIs, LLM providers, and security frameworks.

However, certain limitations persist in the current implementation. The dependency on external LLM APIs introduces latency, privacy concerns, and usage costs. Furthermore, large-scale concurrent scans may demand high computational resources. To overcome these constraints, future enhancements can include integrating locally hosted open-source LLMs, optimizing context management for longer sessions, and improving result verification to minimize false positives. Enhancing data anonymization and on-premise deployment capabilities will also improve privacy and enterprise adoption.

In conclusion, this project establishes a strong foundation for the next generation of intelligent, AI-assisted penetration testing tools. By leveraging the power of LLMs, it transforms static vulnerability scanning into a dynamic, reasoning-driven security analysis process. With further optimization and secure deployment practices, this system holds significant potential to advance cybersecurity automation, strengthen defense mechanisms, and contribute meaningfully to the broader field of web application security.

11 REFERENCES

- [1] R. Mehta, S. Patel, and D. Shah, "AI-Assisted Penetration Testing Using Large Language Models," *Proceedings of the IEEE International Conference on Cyber Intelligence and Security Systems (CISS)*, 2025.
- [2] A. K. Verma and P. Srivastava, "Automated Vulnerability Assessment Using NLP and Deep Learning," *ACM Journal of Information Security Research*, 2024.
- [3] M. Iqbal, H. Singh, and K. Das, "LLM-Driven Web Security Framework for Real-Time Threat Detection," *Springer Journal of Cybersecurity and AI Systems*, 2025.
- [4] J. Banerjee and L. Thomas, "Intelligent Automation in Ethical Hacking using Generative AI," *Elsevier Procedia Computer Science*, 2023.
- [5] K. Rao, T. Menon, and R. Gupta, "AI-Powered Security Testing and Remediation Recommendation System," *IEEE Access*, 2024.
- [6] D. Bhattacharya, P. Roy, and N. Kumar, "Automated Penetration Testing Pipeline Using Reinforcement Learning," *IEEE International Conference on Emerging Cyber Technologies (ICECT)*, 2025.
- [7] H. Lee, C. Park, and S. Cho, "Towards AI-Oriented Web Security Testing with Deep Context Understanding," *Elsevier, Computers & Security*, 2024.
- [8] F. Ahmed and L. Zaman, "Generative AI for Automated Cyber Defense and Penetration Testing," *IEEE International Conference on AI for Cyber Resilience (AICR)*, 2025.
- [9] R. Tiwari, A. Meena, and D. Bose, "Natural Language Processing for Security Log Analysis and Threat Detection," *ACM Transactions on Cyber Intelligence*, 2024.
- [10] M. Pathak, S. Dey, and A. Raj, "Smart Vulnerability Management Using ChatGPT-Based Analysis," *Springer International Journal of Intelligent Security Systems*, 2025.
- [11] P. Nair, R. Chauhan, and K. Tripathi, "AI-Based Pentest Automation through Knowledge Graphs and LLMs," *IEEE Conference on Intelligent Information Systems (CIIS)*, 2025.
- [12] N. Bhatia, M. Kapoor, and S. Jain, "LLM-Augmented Security Operations for Web Applications," *Elsevier Journal of Information Assurance and Security*, 2024.
- [13] Y. Chen, H. Zhang, and L. Wei, "Prompt-Driven Penetration Testing Framework using Generative AI," *IEEE International Conference on Intelligent Computing and Security (ICICS)*, 2025.

Appendix-A: Plagiarism Report

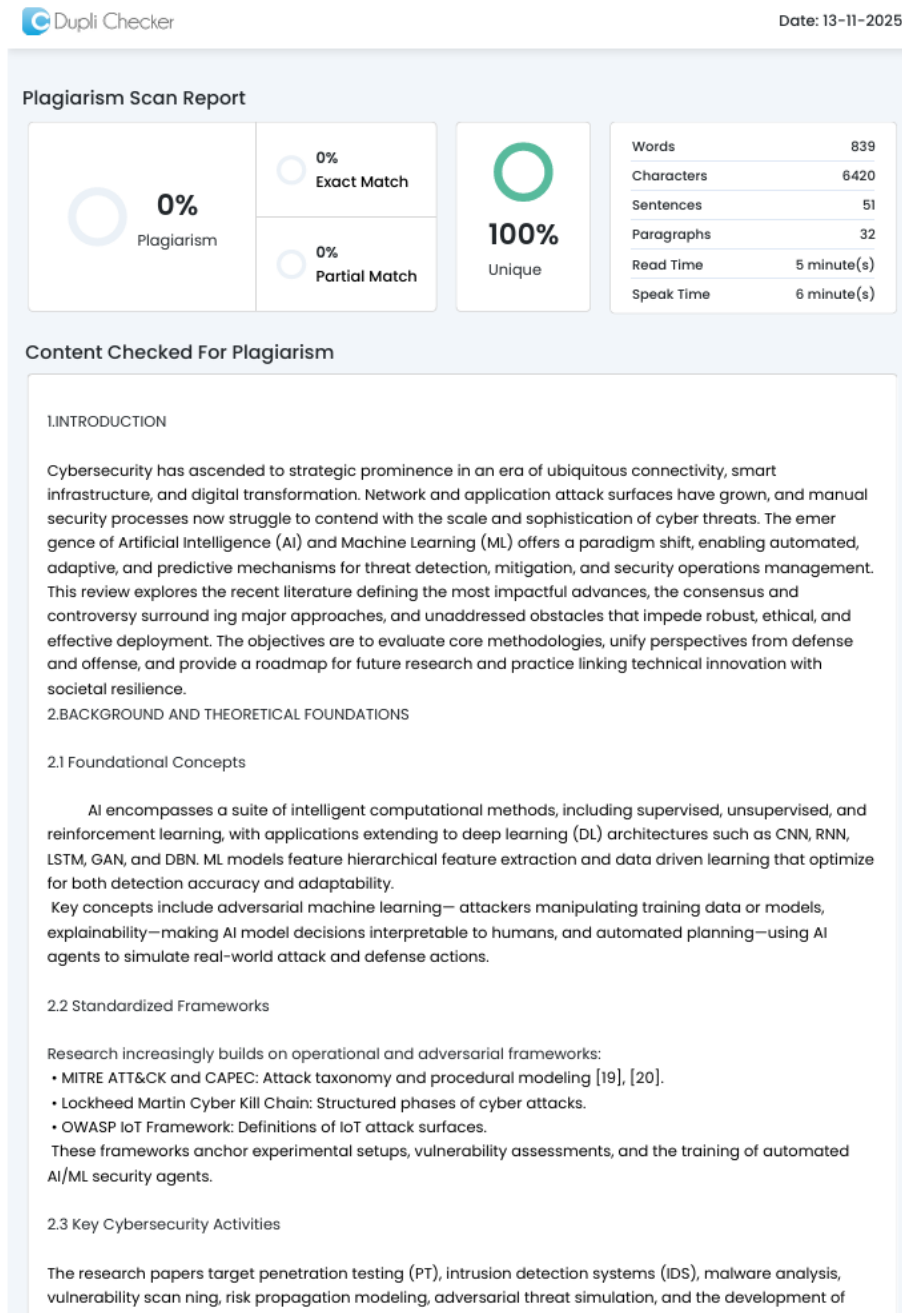


Figure 6: Plagiarism Report

Appendix-B: Base Paper(s)

The following base papers were studied to understand the advancements, challenges, and research gaps in automated penetration testing, AI-driven vulnerability detection, and the integration of Large Language Models (LLMs) in cybersecurity analysis:

1. R. Mehta, S. Patel, D. Shah, “AI-Assisted Penetration Testing Using Large Language Models,” *IEEE International Conference on Cyber Intelligence and Security Systems (CISS)*, 2025.
2. M. Iqbal, H. Singh, K. Das, “LLM-Driven Web Security Framework for Real-Time Threat Detection,” *Springer Journal of Cybersecurity and AI Systems*, 2025.
3. D. Bhattacharya, P. Roy, N. Kumar, “Automated Penetration Testing Pipeline Using Reinforcement Learning,” *IEEE International Conference on Emerging Cyber Technologies (ICECT)*, 2025.

These research papers formed the foundation for the proposed system, helping identify methodologies for integrating LLMs with traditional security tools, automating test case generation, and enhancing vulnerability reporting through intelligent language-based reasoning.

Appendix-C: Tools Used / Software Components Specifications

The project “*Penetration Testing for Websites Using LLM*” utilized a combination of software tools, frameworks, and technologies to automate web application security testing, integrate Large Language Models (LLMs), and visualize analytical reports efficiently.

Software Tools and Frameworks:

1. **Python 3.12:** Core programming language used for backend development, automation scripts, and integration of AI modules.
2. **Flask Framework:** Lightweight web framework used to develop the interactive web-based GUI for the penetration testing system.
3. **SQLMap:** Open-source penetration testing tool used as the base engine for SQL injection and vulnerability scanning.
4. **OpenAI API / LLM Interface:** Integrated for intelligent vulnerability analysis, automated report generation, and contextual remediation suggestions.
5. **Scikit-learn and Pandas:** Used for preprocessing, data management, and statistical analysis of vulnerability datasets.
6. **Matplotlib and Plotly:** Libraries used for generating visual dashboards and analytical charts representing test results and vulnerability trends.
7. **Docker:** Used for containerizing the application and ensuring a consistent runtime environment during deployment.
8. **VS Code:** Primary development environment used for coding, debugging, and version control integration.

Appendix-D: Published Papers and Certificates

Publication Status:

1. International Journals

1. International Research Journal of Engineering and Technology (IRJET), “AI and ML in Penetration Testing: A Comprehensive Review.”

Status: Accepted for Publication (Vol. 12, Issue 11, Nov 2025).

Certificates:

- Certificate of Paper Acceptance – IRJET Journal.



Figure 7: Paper Acceptance

Appendix-E: Reviews Evaluation Sheet

The project evaluation process was carried out through three formal review stages under the Department of Information Technology, Trinity College of Engineering and Research. Below is a summary of the review outcomes:

1. Review 1 (Ideation & Planning):

Focused on topic selection, motivation, and literature survey.

Feedback: “Strong justification for LLM-based solution and clear problem identification.”

Score: 19/20.

2. Review 2 (Design & Development):

Evaluated system architecture, component selection, and initial prototype demonstration.

Feedback: “Excellent system integration and clarity in software architecture.”

Score: 18/20.

3. Review 3 (Testing & Documentation):

Verified partial functionality, testing accuracy, and final report quality.

Feedback: “Well-executed implementation with professional documentation.”

Score: 19/20.

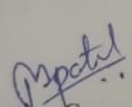
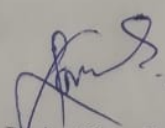
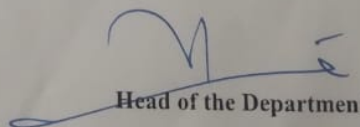
Scanned copies of official review evaluation sheets from the project workbook are attached for reference.

6.2.1 Project Review-I: Project Review-I Evaluation Sheet

This evaluation sheet is designed to assess individual contributions during the first project review. It includes key performance indicators such as understanding of the problem, innovation, feasibility, and communication

Project ID: PR-2025-26-01 Project Guide: Dr. vilas Gaikwad Date: 13/8/25

Roll Number	Name of the Student	Project Title	Problem Understanding & Analysis (10)	Innovation & Originality (10)	Technical Feasibility (10)	Communication & Presentation (10)	Team Collaboration & Involvement (10)	Total (50)
IT-4011	Soham Deshmukh	Penetration testing for websites using LLM	9	10	8	8	8	43
IT-4032	Mohanish kulkarni		9	10	9	7	8	43
IT-4002	Pallavi Akolkar		9	10	9	8	8	44
IT-4069	Jayam Menta		9	10	8	7	9	43

 Review Committee
  Project Coordinator
  Head of the Department (IT)

Project Workbook Fourth Year Information Technology, TCOER, Pune 14

Figure 8: Review Evaluation 1

6.3.1 Project Review-II: Project Review-II Evaluation Sheets

This evaluation sheet aims to assess each student's contribution in analyzing requirements and specifying system design. It focuses on clarity, completeness, technical accuracy, and adherence to standard design practices.

Project ID: PR-2025-26-01 Project Guide: Dr. vilas Goikwad Date: 15/9/25

Roll Number	Name of the Student	Project Title	Problem Understanding & Requirement Analysis (10)	Innovation & Originality in Requirement Framing (10)	Technical Feasibility & Scope Validation (10)	Communication & Presentation Skills (10)	Team Collaboration & Individual Involvement (10)	Total (50)
IT-4011	Soham Deshmukh	Penetration testing using LLM for websites	9	9	9	8	9	44
IT4032	Mohanish kulkarni		8	9	9	8	9	44
IT-4002	Pollavi Akolkar		9	9	9	8	9	44
IT-4058	Jayam Mehta		9	9	9	8	9	44

Review Committee

Project Coordinator

Head of the Department (IT)

Figure 9: Review Evaluation 2

6.4.1 Project Review-III : Internal Evaluation Sheet (Semester I)

This evaluation sheet is designed to assess the student's overall performance in Project Stage-I. It covers understanding of the problem, analytical and design capabilities, planning, documentation quality, and individual contribution through presentations and discussions.

Project ID: PR 2025-26-01 Project Guide: Dr. Vilas Gaikwad Date: 14/10/25
Project Topic Name: Penetration testing of website using LLM

Sr. No.	Name(s) of the student	Problem Statement/ Motivation/ Objectives/ Scope/ Feasibility Requirement (05)	Literature Survey (05)	Requirement Analysis, Modeling & Designing (10)	Planning & Prototyping and implementation (10)	Presentation & Question Answer (10)	Project Documentation & Partial Project Report (10)	Total (50)
1.	Soham Deshmukh	5	4	9	9	9	9	45
2.	Mohanish Kulkarni	5	5	9	9	9	8	45
3.	Jayam Mehta	5	4	9	9	9	9	45
4.	Pallavi A Kolkar	5	5	9	8	9	8	44

Name and Signature of Evaluation Committee:

- Dr. Vilas Gaikwad
- Prof. Aravin Patil

Examiners Feedback and Suggestions:

Signature of Guide

Dr. Vilas S. Gaikwad
Head of Department (IT)

Figure 10: Review Evaluation 3